



Ejercicios Tarea 4

INDICACIONES

- Debe escoger una pareja para intercambiar ideas. Cada integrante debe entregar su tarea de manera individual.
- Esta Tarea consta de 2 ejercicios para evaluar contenidos repasados.
- La Tarea debe ser rendida en la plataforma `http://codetrack.cl/`
- Para entrar debe escribir su correo institucional y como password su rut (solo numeros).
- Lea cada pregunta con atención y programe en la plataforma.
- Cualquier consulta sobre la plataforma debe ser resuelta con los profesores a cargo.
- Puede usar su cheat sheet impreso y entregado en clases.

Contexto

Esta tarea repasa el uso de algoritmos de ordenamiento y búsqueda vistos en la Semana 13, junto con el uso de `np.polyfit` y `np.roots` visto en la Semana 14.

Problemas

1. Ordenamiento y búsqueda en una lista de periodos orbitales.

Enunciado

En una observacion simplificada se registraron los periodos orbitales, en dias, de un conjunto de exoplanetas:

```
periodos = [12.4, 3.1, 8.7, 25.0, 1.9, 14.2, 6.5, 18.3].
```

Utilice los siguientes algoritmos ya dados. No debe construirlos desde cero:

```
1 def bubble_sort(lista):
2     n = len(lista)
3     for i in range(n - 1):
4         for j in range(n - 1 - i):
5             if lista[j] > lista[j + 1]:
6                 lista[j], lista[j + 1] = lista[j + 1], lista[j]
7     return lista
8
9 def binary_search(lista_ordenada, objetivo):
10     low, high = 0, len(lista_ordenada) - 1
11
12     while low <= high:
13         mid = (low + high) // 2
14
15         if lista_ordenada[mid] == objetivo:
16             return mid
17         elif objetivo < lista_ordenada[mid]:
18             high = mid - 1
19         else:
20             low = mid + 1
21
22     return -1
```

Realice las siguientes operaciones:

- (a) Defina la lista `periodos`.
- (b) Ordene los periodos de menor a mayor usando `bubble_sort`. Use una copia de la lista original para no modificarla directamente.
- (c) Imprima la lista original y la lista ordenada.
- (d) Determine el periodo minimo y el periodo maximo usando la lista ordenada.
- (e) Use `binary_search` para buscar el periodo `14.2` en la lista ordenada e imprima su posicion.
- (f) Use `binary_search` para buscar el periodo `10.0`. Indique con un mensaje si no fue encontrado.
- (g) Explique brevemente, dentro de un `print`, por que la busqueda binaria debe aplicarse sobre una lista ordenada.

2. Ajuste lineal y estimacion de un umbral de temperatura.

Enunciado

Durante el enfriamiento de una muestra se midio la temperatura T , en grados Celsius, en distintos tiempos t , en minutos:

$$t = [0, 2, 4, 6, 8, 10],$$
$$T = [82, 70, 59, 48, 38, 29].$$

Suponga que, en este intervalo, la temperatura puede aproximarse por un modelo lineal:

$$T(t) = mt + b.$$

Realice las siguientes operaciones:

- (a) Importe `numpy` y `matplotlib.pyplot`. Defina los arreglos `t` y `T` usando `np.array`.
- (b) Use `np.polyfit(t, T, deg=1)` para encontrar la pendiente m y el intercepto b .
- (c) Calcule los valores ajustados y guárdelos en una variable llamada `T_fit`.
- (d) Calcule los residuos y el coeficiente R^2 . Para esto use:

$$\text{residuos} = T - T_{\text{fit}},$$

$$SS_{\text{res}} = \sum_i (T_i - T_{\text{fit},i})^2,$$

$$SS_{\text{tot}} = \sum_i (T_i - \bar{T})^2,$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

- (e) Grafique los datos medidos con `plt.scatter` y el ajuste lineal con `plt.plot`. Agregue etiquetas, leyenda y grilla.
- (f) Use `np.roots` para estimar en que tiempo el modelo ajustado alcanza $T = 35^\circ\text{C}$. Para ello resuelva:

$$mt + b - 35 = 0.$$

- (g) Imprima m , b , R^2 y el tiempo estimado para alcanzar 35°C . interprete brevemente dentro de un `print` el signo de m .